POLITECHNIKA RZESZOWSKA
im. IGNACEGO ŁUKASIEWICZA

KATEDRA INFORMATYKI I AUTOMATYKI
POLITECHNIKI RZESZOWSKIEJ

Renderowanie i wydajność aplikacji XR

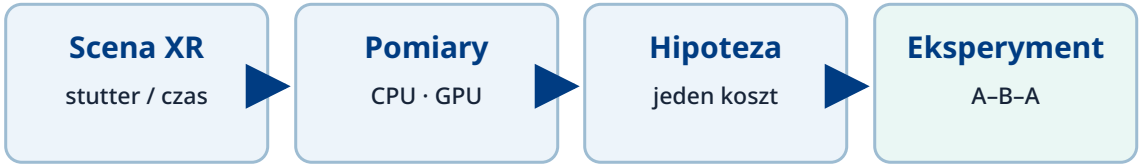
WiRR · Wykład 06 · Automatyka i Robotyka · 3 ECTS · 2026/2027 · v1.1.0

dr inż. Mateusz Pomianek

Katedra Informatyki i Automatyki, Politechnika Rzeszowska

27.10.2026

Cel: znaleźć koszt klatki przed obniżaniem jakości

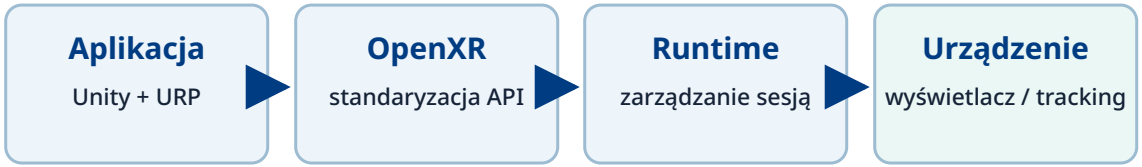


Nie zmieniaj jakości bez identyfikacji ograniczenia

Porównaj **tę samą scenę XR** przed i po jednej zmianie. Nie obniżaj jakości, dopóki nie rozpoznasz wąskiego gardła.

- Czy termin klatki przekracza CPU, GPU czy synchronizacja?
- Jak wyglądają rozkłady czasów po rozgrzewce?
- Czy użytkownik nadal odczytuje etykiety i wykonuje zadanie?

OpenXR: wspólne API



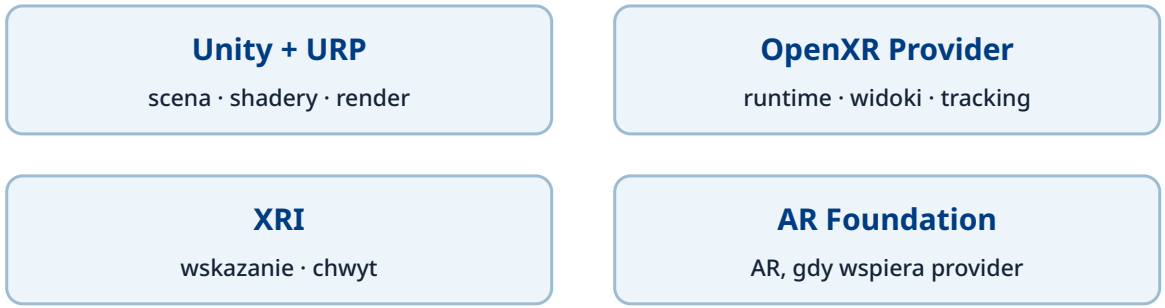
Wspólne API nie wyrównuje funkcji sprzętowych

OpenXR standaryzuje interfejs aplikacji z runtime XR, ale nie ujednolica możliwości urządzeń.

`Aplikacja → OpenXR → Runtime → Zestaw XR`

Sprawdzaj osobno: obsługiwane rozszerzenia, tryb renderowania, dostępność śledzenia oraz docelową częstotliwość. Wspólne API nie oznacza jednakowego FPS.

Unity, XRI i AR Foundation



Każda warstwa rozwiązuje inny problem

- **Unity + URP:** logika sceny, shadery i rendering.
- **XR Plug-in Management + OpenXR:** integracja z runtime i widokami XR.
- **XR Interaction Toolkit:** wskazywanie, wybór, chwyt.
- **AR Foundation:** płaszczyzny, kotwice i śledzenie, jeżeli obsługuje je provider.

Gdy spada FPS, najpierw profiluj rendering, a nie wymieniaj komponentów interakcji.

Aplikacja, runtime i kompozytor: kto odpowiada za obraz?

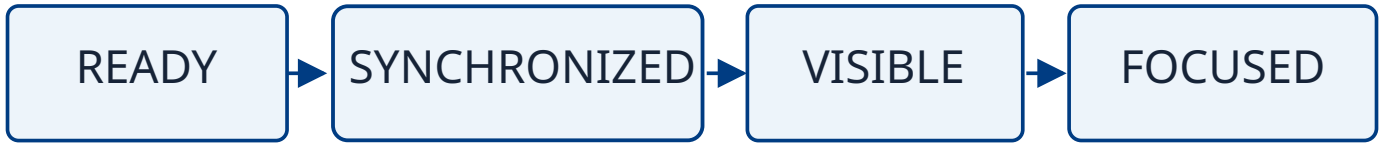


Reprojekcja nie generuje nowego stanu sceny

Aplikacja renderuje nowe widoki sceny. **Runtime** planuje pracę z urządzeniem. **Kompozytor** prezentuje warstwy, niekiedy korygując je na podstawie nowszej pozy.

Stabilna prezentacja po reprojekcji **nie oznacza**, że aplikacja wygenerowała nową klatkę. Rozróżniaj licznik klatek aplikacji i zdarzeń prezentacji.

Sesja OpenXR: działanie nie oznacza widoczności

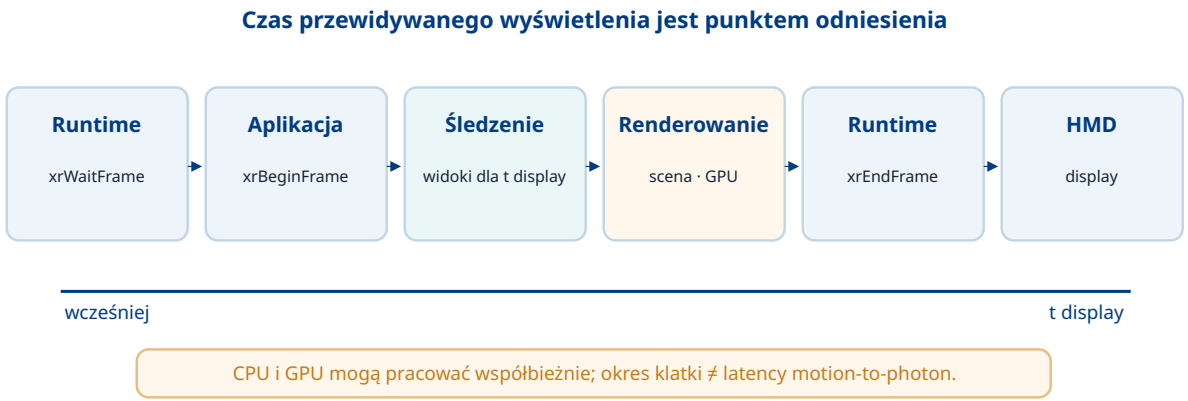


STOPPING: zakończenie sesji

Sesja `SYNCHRONIZED` może nie być jeszcze widoczna. `VISIBLE` oznacza możliwość prezentacji, a `FOCUSED` również fokus wejścia.

Diagram pokazuje **typowy fragment cyklu**, nie wszystkie legalne przejścia stanów ani zalecaną uniwersalną maszynę stanów. Nie łącz aktywności wejścia z samym istnieniem sesji.

Pętla klatki odnosi się do czasu wyświetlenia



OpenXR dostarcza **przewidywany czas prezentacji**; widoki i pozycję oczu wyznaczamy względem niego.

`xrWaitFrame → xrBeginFrame → xrLocateViews → render → xrEndFrame`

Klatka jest przygotowywana dla spodziewanego momentu wyświetlenia, a nie dla przypadkowego momentu pobrania wejścia.

Motion-to-photon: FPS nie wyczerpuje opisu opóźnienia



Cały łańcuch ≠ 1000 / FPS

Motion-to-photon obejmuje drogę od ruchu głowy do zmiany światła docierającego z wyświetlacza.

`Sensor → estymacja pozy → aplikacja → GPU → kompozytor → wyświetlacz`

`90 Hz = 11,11 ms` oznacza **okres odświeżania**, a nie całkowitą latencję MTP. Reprojekcja ogranicza niektóre skutki opóźnienia, lecz nie dostarcza brakujących nowych zmian sceny.

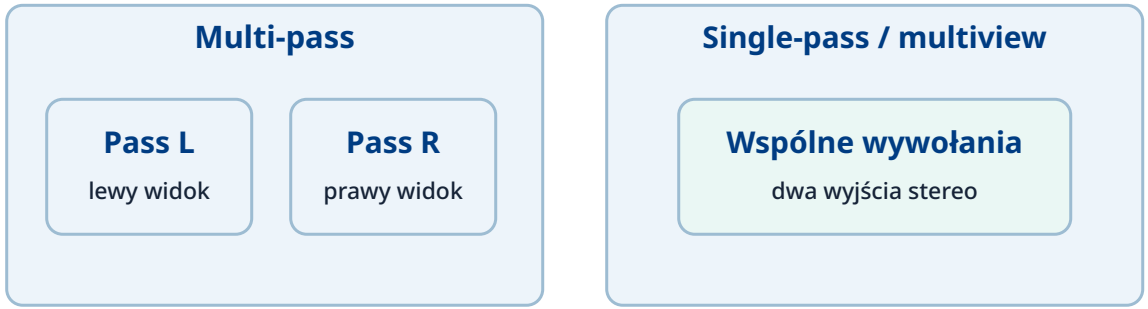
Rendering stereoskopowy



Dwa oczy wymagają **różnych pozycji widoku i projekcji**. Prawidłowa geometria stereoskopowa jest warunkiem jakości i komfortu.

Rendering obu oczu nie musi kosztować dokładnie dwa razy tyle co pojedynczy widok: część pracy może być współdzielona przez silnik i GPU.

Single Pass Instanced i multiview: mniej powtarzanej pracy



Oszczędność zależy od platformy i shaderów

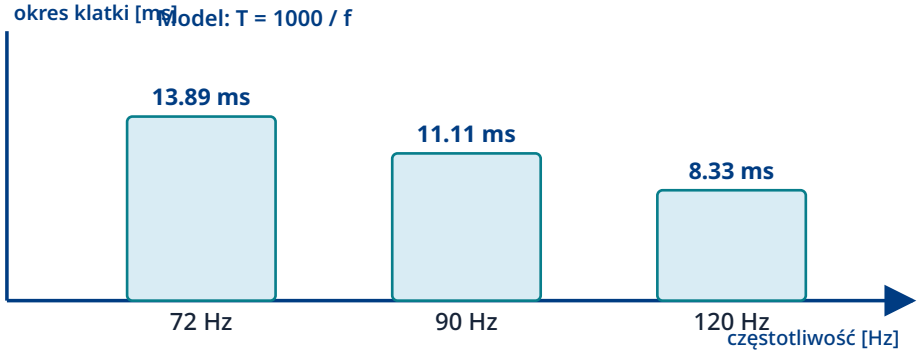
Multi-pass: osobne przejścia dla dwóch widoków. **Single Pass Instanced / multiview:** ograniczenie części powtarzalnej pracy związanej z przygotowaniem renderowania.

Oszczędność zależy od platformy, API grafiki i shaderów; **oba obrazy nadal wymagają przetwarzania pikseli**.

Single Pass Instanced i multiview: mniej powtarzanej pracy

Unity: `Project Settings → XR Plug-in Management → OpenXR → Render Mode`, jeżeli provider udostępnia tę opcję.

Budżet klatki



Czas jednego okresu wyświetlania wynosi

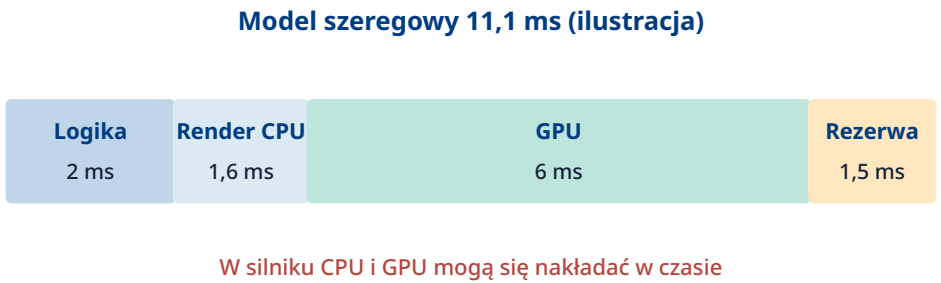
$$T = \frac{1}{f}, \qquad T_{\mathrm{ms}} = \frac{1000}{f_{\mathrm{Hz}}}.$$

- **72 Hz:** 13,89 ms
- **90 Hz:** 11,11 ms
- **120 Hz:** 8,33 ms

Budżet klatki

To **termin dostarczenia klatki**, nie wynik pomiaru CPU/GPU ani motion-to-photon.

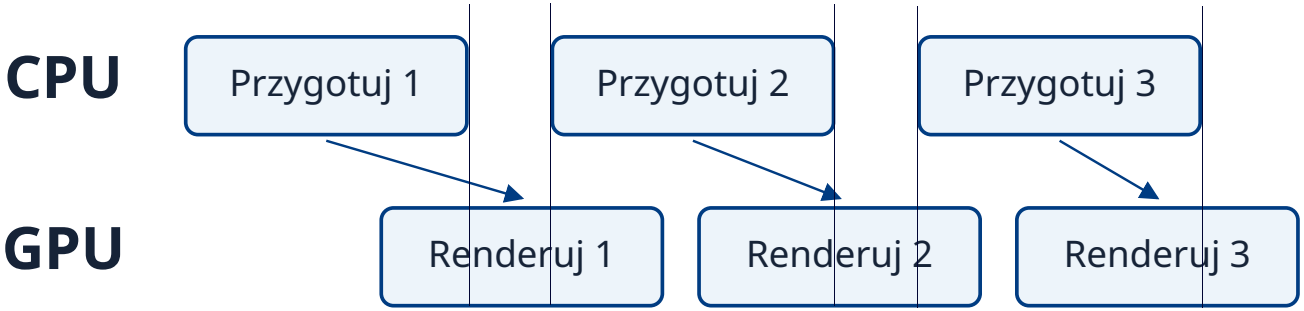
Jak interpretować budżet klatki



Przykład modelowy szeregowy dla 90 Hz: logika 2,0 ms + przygotowanie renderowania 1,6 ms + GPU 6,0 ms + rezerwa 1,5 ms = **11,1 ms**.

To uproszczenie dydaktyczne. W rzeczywistym silniku CPU i GPU wykonują pracę częściowo równolegle; **sumowanie liczników różnych kolejek jest błędne**. Ustal, który licznik i zakres czasu opisuje dana wartość.

CPU i GPU: równoległość zmienia rachunek



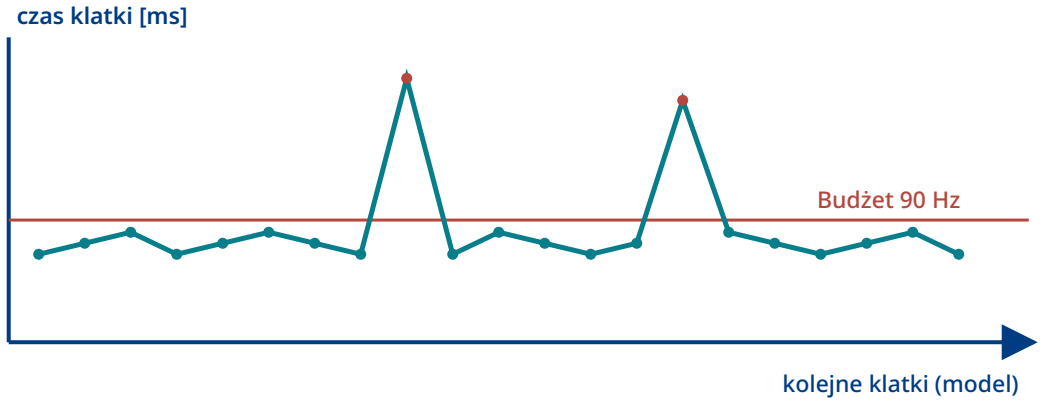
Praca kolejnych klatek może nakładać się w czasie

CPU może przygotowywać klatkę $n+1$, gdy GPU renderuje klatkę n.

Przepustowość w stanie ustalonym zależy od ograniczającego etapu i synchronizacji. **Latencja pojedynczej klatki** nie jest równa odstępowi między gotowymi klatkami.

Jeżeli CPU oczekuje na GPU, nie oznacza to automatycznie, że skrypt jest wolny.

Jedna liczba FPS ukrywa niestabilność



W dwóch przejazdach mediana może być identyczna, lecz **p95** i odsetek przekroczeń budżetu znacznie się różnią.

- Raportuj **medianę, p95 i liczbę klatek powyżej budżetu**.
- Podaj liczbę próbek, czas rozgrzewki i scenariusz ruchu.
- Wykres jest **modelem**, nie pomiarem konkretnego headsetu.

Jitter, zacięcia i pominięte klatki



Jitter oznacza nieregularność odstępów między klatkami; **stutter** to widoczne zacięcie związane z takim zakłóceniem.

Rozróżniaj: spóźnioną klatkę aplikacji, brak nowej klatki oraz prezentację obrazu skorygowanego przez runtime. Średni FPS nie identyfikuje tych zjawisk.

Foveated rendering



Największą szczegółowość zachowujemy w obszarze najistotniejszym dla widzenia.

- **Fixed foveation:** strefa wyższej jakości jest związana z pozycją w widoku.
- **Eye-tracked foveation:** strefa może podążać za oszacowanym spojrzeniem.

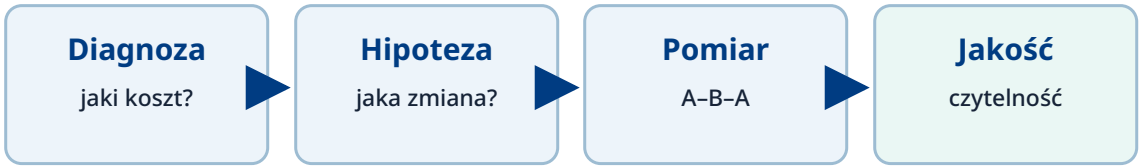
Wariant drugi zależy od jakości kalibracji i opóźnienia eye trackingu; żadnego nie traktuj jako gwarancji przyspieszenia.

Przerwa 5 minut

Podsumowanie pierwszej części: **okres odświeżania to nie latencja**, a poprawa FPS bez wskazania ograniczającego etapu może być pozorna.

Po przerwie przechodzimy do eksperymentów z geometrią, rozdzielczością i kosztami pikselowymi.

Optymalizacja



Zmierz ograniczenie, zanim obniżysz jakość

Reguła eksperymentu: diagnoza → hipoteza → jedna zmiana → pomiar → kontrola jakości.

Przykładowa hipoteza: *przezroczyste nakładki zwiększają koszt pracy na pikselach*. Wtedy testuj ich usunięcie, zamiast redukować trójkąty w całej scenie.

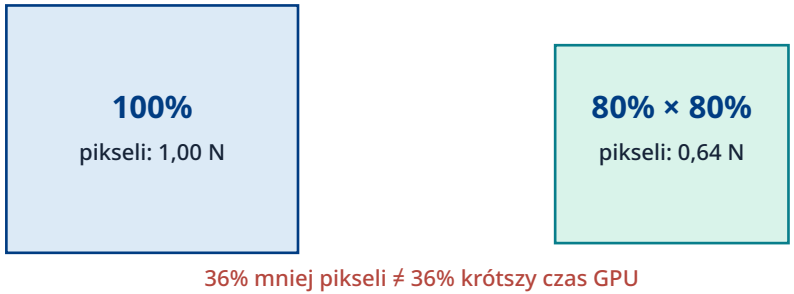
Unity, warianty jednej sceny:

Optymalizacja

```
public void ShowBaseline() => Apply(true, true);
public void ShowLowDetail() => Apply(false, true);
```

Pełny demonstrator C#.

Dynamic resolution: kiedy obniżanie rozdzielczości pomaga?



Zmniejszenie liniowej rozdzielczości renderowania do **80%** daje **64%** pierwotnej liczby pikseli, zakładając skalowanie obu osi.

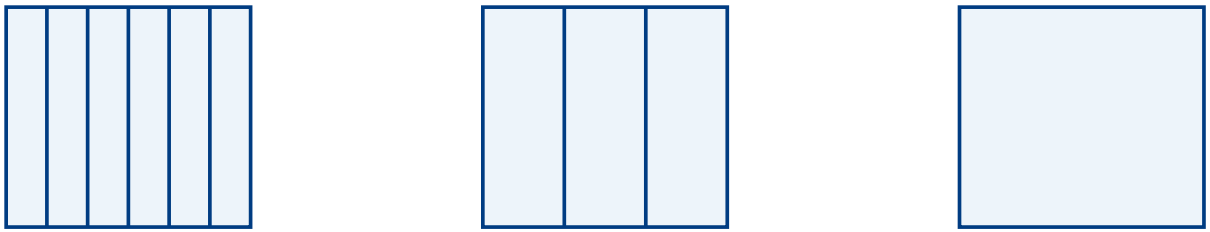
$$N'/N = 0,8^2 = 0,64$$

Może pomóc przy ograniczeniu pikselowym GPU, lecz nie skraca automatycznie czasu skryptów CPU. **Sprawdź ostrość etykiet XR.**

Dynamic resolution: kiedy obniżanie rozdzielczości pomaga?

Nie wszystkie konfiguracje XR obsługują dynamiczne skalowanie w jednakowy sposób.

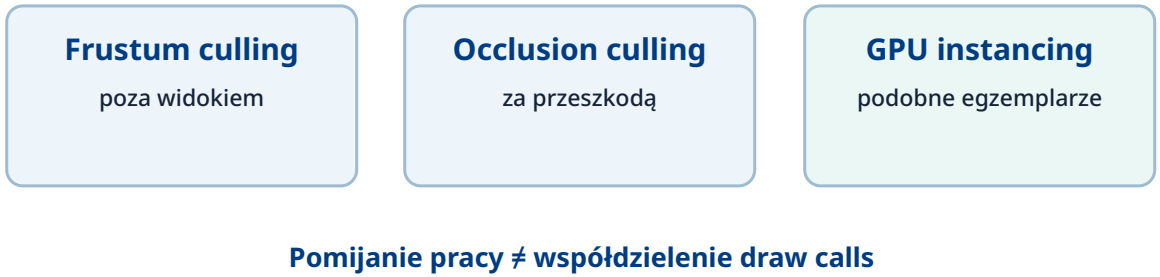
LOD zmienia geometrię, nie znaczenie zadania



LOD: wybór uproszczonej siatki wraz z odległością lub wielkością projekcji.

Zmniejszenie liczby trójkątów pomaga tylko wtedy, gdy odciąża **rzeczywisty koszt ograniczający**. Element krytyczny dla zadania, np. dźwignia czy przycisk, musi pozostać rozpoznawalny.

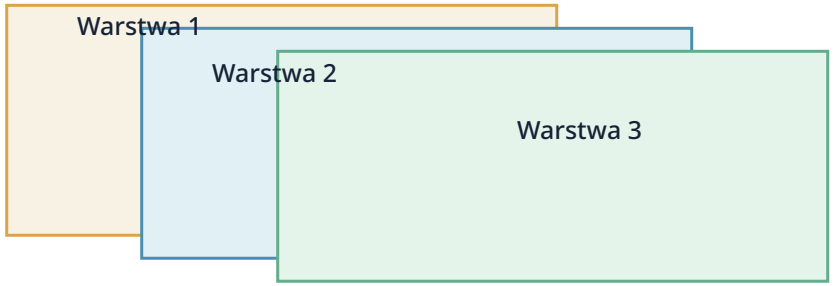
Culling i instancing rozwiązują inne problemy



- **Frustum culling:** nie renderuj obiektu poza bryłą widzenia.
- **Occlusion culling:** pomijaj zasłonięte obiekty.
- **GPU instancing:** ogranicz powtarzaną pracę przy zgodnych instancjach siatki i materiału.

W URP wynik instancingu zależy od shaderów i SRP Batcher. **Mierz liczbę draw calls i koszt CPU**, zamiast zakładać przyspieszenie.

Fill rate, overdraw i koszt przezroczystości



Wspólny obszar → powtarzana praca pikselowa

Overdraw występuje, gdy ten sam obszar ekranu jest cieniowany lub zapisywany wielokrotnie.

Duży półprzezroczysty panel może obciążać GPU mimo **małej liczby trójkątów**. Przy przezroczystości istotne są rozmiar powierzchni na ekranie, shader i blendowanie.

Fill rate, overdraw i koszt przezroczystości

Próba w Unity: wyłącz tylko nakładki przezroczyste i porównaj odpowiednią metrykę GPU, gdy jest dostępna.

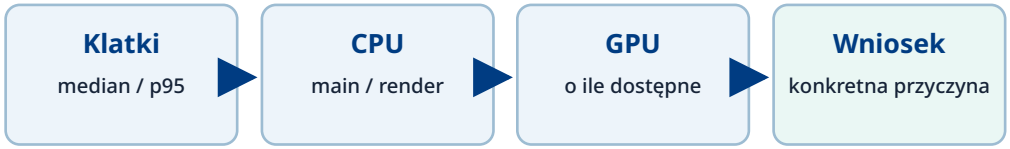
Kod: budżet jest progiem, nie benchmarkiem

Okres 90 Hz: `11,11 ms`. Porównanie ma sens dopiero po ustaleniu znaczenia licznika.

```
float budgetMs = 1000f / targetHz;
bool overBudget = mainThreadMs > budgetMs;
```

`mainThreadMs` oznacza **czas głównego wątku CPU**, nie całkowity czas klatki ani koszt GPU. Zobacz rejestrator Unity.

Profiler: diagnoza przed zmianą



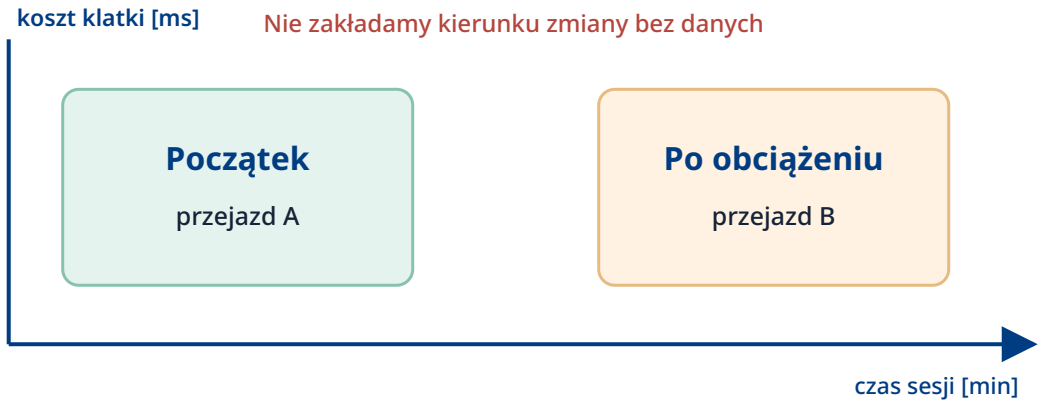
Wait nie jest automatycznie dowodem wolnego kodu

Kolejność: identyfikuj odcinek trasy z przekroczeniami → rozdziel CPU, render thread, GPU i synchronizację → dopiero wtedy optymalizuj komponent.

```
// ProfilerRecorder: nanosekundy -> milisekundy
double mainMs = recorder.LastValue / 1_000_000.0;
```

Ważne: etykieta `Wait` nie identyfikuje przyczyny bez analizy zależności. GPU timing może być niedostępny na danej platformie.

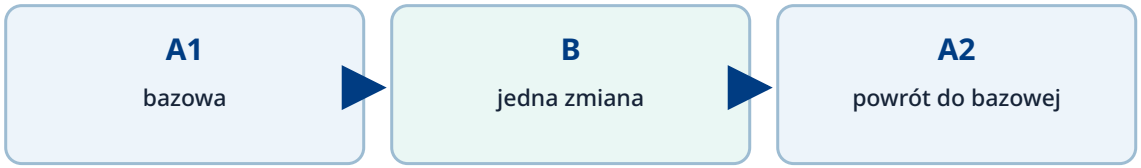
Termika i stabilność po dłuższej pracy



Pomiar tuż po uruchomieniu ≠ pomiar po długim obciążeniu. Sprzęt może ograniczać częstotliwości pracy z powodów termicznych lub energetycznych.

Porównaj identyczny przejazd na początku i później. Rejestruj warunki środowiska, czas testu, stan baterii i dostępne metryki termiczne. Bez nich nie przypisuj zmiany wydajności do temperatury.

Kontrolowane porównanie wydajności



Porównuj p95 i jakość, nie tylko średnie FPS

A–B–A: stan bazowy → jedna zmiana → powrót do stanu bazowego.

- Stały build, trasa, rozdzielczość, urządzenie i tryb zasilania.
- Rozgrzewka, porównywalne okna i liczba próbek.
- Raportuj medianę, p95, przekroczenia budżetu i jakość zadania.

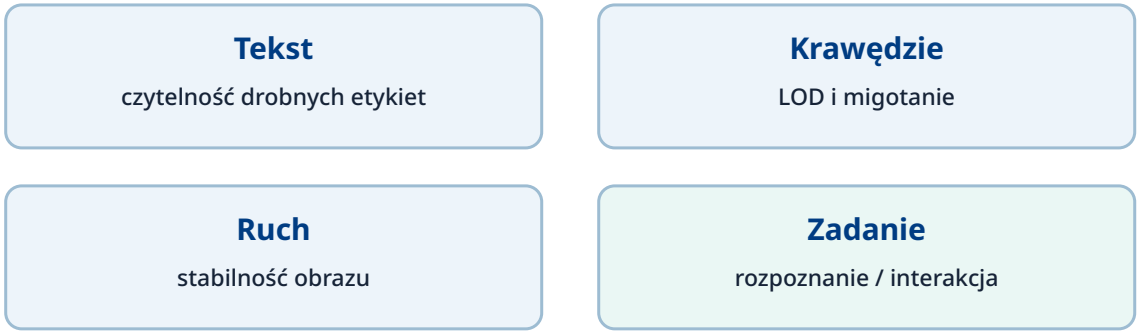
Różnicę zaakceptuj dopiero wtedy, gdy da się ją powtórzyć.

Wydajność i doświadczenie użytkownika

Skrócenie czasu GPU o 2 ms jest użyteczne tylko wtedy, gdy **nie narusza wykonania zadania**.

Mierz dwie grupy wskaźników: **techniczne** (czas, p95, stutter) i **percepcyjne** (czytelność tekstu, rozpoznawanie obiektów, komfort). Nie utożsamiaj ich ze sobą.

Jakość percepcyjna: co warto sprawdzać po optymalizacji?



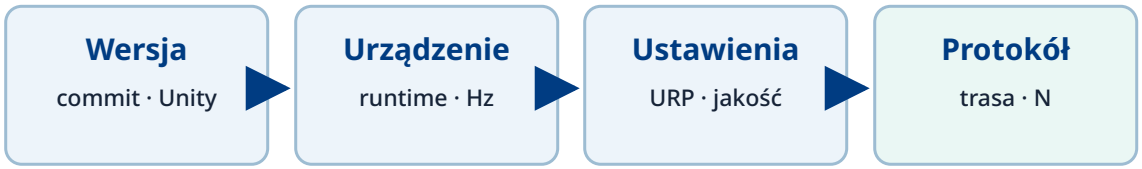
Po zmianie LOD, foveation lub rozdzielczości sprawdź w **ruchu**, a nie tylko na jednym zrzucie:

- Czy drobne etykiety są czytelne?
- Czy sylwetka i krawędzie istotne dla zadania nie migoczą?
- Czy elementy interaktywne pozostają identyfikowalne?
- Czy jakość jest porównywalna między oczami?

Jakość percepcyjna: co warto sprawdzać po optymalizacji?

Zysk czasu bez jakości funkcjonalnej nie jest pełnym sukcesem.

Reprodukowanie projektu



Same FPS bez kontekstu nie są porównywalne

Wynik zależy od **urządzenia, wersji Unity, URP, OpenXR, assetów, shaderów, jakości i trasy**.

Zapisuj commit, platformę, konfigurację renderowania, liczbę próbek, warunki i definicję każdego licznika. Bez tego dwóch wyników „90 FPS“ nie można rzetelnie porównać.

Podsumowanie

1. Ustal częstotliwość i **budżet** okresu klatki.
2. Zidentyfikuj ograniczenie **CPU / GPU / synchronizacji**.
3. Dobierz jedną technikę: LOD, culling, instancing, foveation lub rozdzielczość.
4. Porównaj **medianę, p95 i błędy**, a następnie **jakość zadania**.

Zasada: optymalizujemy ograniczający koszt, nie liczbę trójkątów dla samej liczby.

Przykładowe pytania egzaminacyjne

1. Dlaczego przy 90 Hz dostępne jest 11,11 ms, lecz motion-to-photon może być inne?
2. Czym różnią się multi-pass, single-pass instanced i multiview?
3. Kiedy obniżenie rozdzielczości nie poprawi wydajności?
4. Jak przygotować wiarygodne porównanie A–B–A po optymalizacji?

Bibliografia

- Khronos, OpenXR 1.1 – specyfikacja sesji i pętli klatki.
- Unity 6000.6, ProfilerRecorder – Frame Timing Manager.
- Unity, Stereo rendering – single-pass i multiview.
- Unity, Profilowanie wydajności grafiki.